

# **Options Backtesting Frameworks**

## **Executive Summary**

Options trading requires specialized backtesting frameworks to simulate multi-leg strategies, option chains, and risk measures such as Greeks and implied volatility. Standard equity backtesting frameworks often cannot handle the complexity of options, making framework selection critical for research and live deployment.

This project evaluated four Python-based frameworks: Backtrader, QuantConnect LEAN, Zipline, and VectorBT. Assessment was based on literature review and documentation, focusing on ease of use and documentation, options support, flexibility and customization, and community resources. A simple naked call strategy was implemented in QuantConnect LEAN to demonstrate the practical execution of an options strategy.

Findings from this project will help identify the most suitable backtesting framework for options strategies, enabling rigorous evaluation, optimization, and scenario testing. This project concludes with recommendations for an options backtesting framework tailored to different scenarios.

## **Methodology**

This project examines four major Python-based backtesting frameworks for options: Backtrader, QuantConnect LEAN, Zipline, and VectorBT. Each framework was evaluated primarily through literature review and documentation analysis, focusing on four key criteria: ease of use and documentation (installation, tutorials, and API clarity), options support (instrument coverage, option chain handling, Greeks and IV data, multi-leg capabilities, and realistic execution simulation), flexibility and customization (ability to implement complex strategies and extend system internals), and community resources (developer activity, support channels, and third-party integrations).

To validate the evaluation approach, the following naked call strategy was implemented and tested in QuantConnect LEAN: a long ATM naked call on NVDA with the farthest expiry, one contract at a time, and held until expiry. Findings from the framework evaluation informed scenario-based guidance for selecting the most suitable backtester depending on the user's needs and objectives.

## Results

Backtrader is a beginner-friendly, locally deployable framework with extensive documentation and broker connectivity. It handles OHLC time-series data for individual contracts, but lacks native multi-strike and multi-expiry options support. Users must load each contract as a separate DataFeed and manually implement complex logic such as spreads, rolling positions, Greeks, and implied volatility. Its flexible, event-driven architecture supports custom order lifecycles, and a large historical user base provides solutions to common problems. The main drawback is the absence of built-in options tools.

| Framework         | Ease of Use & Documentation                                                 | Options Support                                              | Flexibility & Customization                 | Community & Maintenance                                         |
|-------------------|-----------------------------------------------------------------------------|--------------------------------------------------------------|---------------------------------------------|-----------------------------------------------------------------|
| Backtrader        | <a href="#">High</a> — Detailed docs, Easy setup, Local, Broker-Connectable | Limited — Only OHLC time-series, No multi-strike expirations | Medium — Customizable but manual strategies | Sufficient — Active forums, Community will return in the future |
| QuantConnect LEAN | <a href="#">High</a> — Solid platform, Complete API docs, Cloud             | Robust — Automated data handling and backtest                | High — Adjustable parameters                | Strong — Active development and community                       |

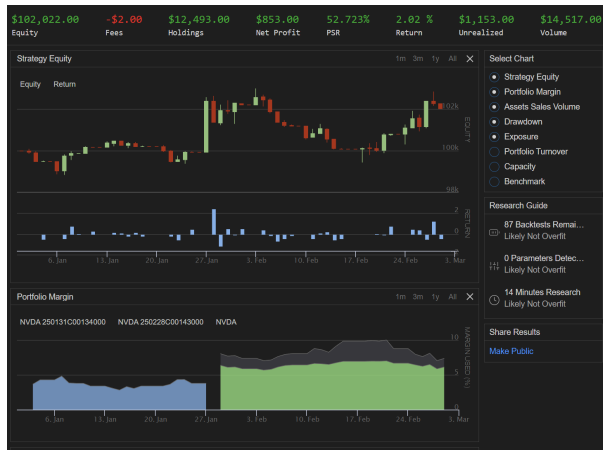
|          |                                                                                                                            |                                                             |                                                                                  |                                                           |
|----------|----------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|----------------------------------------------------------------------------------|-----------------------------------------------------------|
|          | tooling and local,<br>Broker Integrated                                                                                    | execution,<br>Built-in<br>statistics and<br>visuals         |                                                                                  |                                                           |
| Zipline  | <a href="#">Medium</a> — Decent<br>docs, Local,<br>Limited broker<br>integration                                           | Limited —<br>Options require<br>heavy<br>customization      | Medium —<br>Customizable but<br>manual strategies                                | Moderate —<br>Smaller<br>community,<br>Infrequent updates |
| VectorBT | <a href="#">High</a> — Modern<br>and detailed API,<br>Pythonic, Local,<br>Good tutorials,<br>Limited broker<br>integration | Limited —<br>Options<br>modeled as<br>vectorized<br>payoffs | High — Fully<br>vectorized,<br>Customizable<br>portfolio or<br>strategy modeling | Growing — Active<br>community on<br>GitHub and<br>Discord |

QuantConnect LEAN provides a well-structured API, thorough documentation, and robust broker integration, available both locally and in the cloud. It supports options chains, multi-leg strategies, Greeks, implied volatility, expiry handling, and assignment/exercise logic. Automated backtesting, built-in statistics, and flexibility across asset classes make it ideal for production-ready options strategies. Advanced features and enterprise-level data may require a paid plan, and implementations are tied to the QuantConnect ecosystem.

```

3 class NakedCallAlgorithm(QCAlgorithm):
4     def initialize(self) -> None:
5         self.set_start_date(2015, 1, 1)
6         self.set_end_date(2025, 3, 1)
7         self.set_cash(100000)
8
9         option = self.add_option("WDA")
10        self.symbol = option.symbol
11        option.set_filter(lambda universe: universe.include_weeklys().naked_call(30, 0))
12
13        self.call = None
14
15        # use the underlying equity as the benchmark
16        self.set_benchmark(self.symbol.underlying)
17
18    def on_data(self, slice: Slice) -> None:
19        if self.call and self.portfolio[self.call].invested:
20            return
21
22        chain = slice.option_chains.get(self.symbol)
23        if not chain:
24            return
25
26        # Find ATM call with the farthest expiry
27        expiry = max([x.expiry for x in chain])
28        call_contracts = sorted([x for x in chain
29                                if x.right == OptionRight.CALL and x.expiry == expiry],
30                                key=lambda x: abs(chain.underlying.price - x.strike))
31
32        if not call_contracts:
33            return
34
35        atm_call = call_contracts[0]
36
37        naked_call = OptionStrategies.naked_call(self.symbol, atm_call.strike, expiry)
38        self.buy(naked_call, 1)
39
40        self.call = atm_call.symbol

```



|                       |                     |                             |             |
|-----------------------|---------------------|-----------------------------|-------------|
| PSR                   | 52.723%             | Sharpe Ratio                | 0.494       |
| Total Orders          | 3                   | Average Win                 | 0.86%       |
| Average Loss          | 0%                  | Compounding Annual Return   | 13.184%     |
| Drawdown              | 3.380%              | Expectancy                  | -1          |
| Start Equity          | 100000              | End Equity                  | 102022      |
| Net Profit            | 2.022%              | Sortino Ratio               | 0.955       |
| Loss Rate             | 100%                | Win Rate                    | 0%          |
| Profit-Loss Ratio     | 0                   | Alpha                       | 0.014       |
| Beta                  | -0.135              | Annual Standard Deviation   | 0.079       |
| Annual Variance       | 0.006               | Information Ratio           | 0.342       |
| Tracking Error        | 0.659               | Treynor Ratio               | -0.291      |
| Total Fee             | \$2.00              | Estimated Strategy Capacity | \$100000.00 |
| Lowest Capacity Asset | NVDA 25013100134000 | Portfolio Turnover          | 0.24%       |
| Drawdown Recovery     | 12                  |                             |             |
| Rolling Statistics    |                     |                             |             |
|                       | 1 Month             | 3 Months                    | 6 Months    |
| 01/31/2025            | 1.1212              | 1.1212                      | 1.1212      |
| 02/28/2025            | -0.3558             | 0.8019                      | 0.8019      |

Zipline was designed for equities and ETFs, with minimal options capabilities. Its documentation is adequate, but setup can be tricky due to dependencies. Broker integration is limited, and almost all options-specific features—including chains, Greeks, and expiry handling—must be implemented manually. While Zipline is open-source and works offline, its ecosystem has shrunk since official development slowed, with activity shifting to community forks. It's most suitable for users who already have pricing and data handling systems in place and just need a transparent engine to run them.

VectorBT is a Pythonic, fully vectorized backtesting library optimized for high-speed local research. Options are modeled as vectorized payoffs, making it excellent for payoff analysis, scenario testing, and prototyping large parameter sweeps. This approach makes it extremely fast for running large-scale scenario tests, such as scanning across strikes, expiries, and parameter combinations. Users can easily integrate their own pricing models, such as Black-Scholes, and produce risk/reward heatmaps efficiently. However, it lacks native event-driven execution and broker integration, so rolling positions, assignment, and execution delays must be modeled manually. It is ideal for research rather than live trading.

## Recommendations

For comprehensive options backtesting, QuantConnect LEAN is the go-to framework. Its workflow integrates options chain retrieval, built-in Greeks, implied volatility, expiry handling, multi-leg strategies, and automated backtesting, with both cloud and local

deployment and strong broker integration, all available in just one click. Thus, it acts as a complete, production-ready backtesting and execution environment. While some advanced features require paid data or cloud usage, its structured API and active community make it the most reliable option for full-scale strategy development.

Backtrader is recommended for users who need hands-on control of event-driven strategies and may want to integrate with brokers locally. Its flexible architecture, beginner-friendly setup, and extensive documentation make it suitable for custom or unconventional options strategies, though native options support is limited and requires manual implementation.

On the other hand, VectorBT is ideal for rapid, large-scale scenario testing, such as scanning multiple strikes, expiries, and parameters for payoff modeling and risk/reward analysis. Its fully vectorized, Pythonic design makes it extremely fast, but it lacks event-driven execution and broker connectivity, so it is best suited for research and prototyping rather than live trading.

In summary, QuantConnect LEAN should be the default for most due to its comprehensive capabilities and automated workflow, Backtrader for fully customizable execution, and VectorBT for fast, large-scale scenario analysis.